

WORKSHEET

Sensors, Noizetoyz & Toons Handout

Your reference for three parts of today's hacking session — task checklists, checkpoints, and troubleshooting, so you're not stuck reading a screen the whole time. Work through these at your own pace, in any order, on just a laptop (some hardware is a bonus, not a requirement). Print double-sided; each section starts on its own page.

1. 📡🌡️ Live Data Streaming — sensor tasks
2. 🎹🎵 Noizetoyz — play a live synth
3. 🏠🔥 Airglow Toons — build a custom feed automation
4. Appendix — OLED status icons & DHT22 sensor wiring

📡🌡️ LIVE DATA STREAMING

Sensor Tasks

A Raspberry Pi or Wemos board + sensor writes readings as ATProto records via **Nebra**; a separate consumer reads them back out, Matadisco-style. No hardware needed to try this — the producer simulates a reading by default. `cd live-data` for everything below.

Setup

Python ≥3.11

- ☐ `pipenv install && pipenv shell` (or `pip install nebra` directly)
- ☐ Set your identity: `NEBRA_HANDLE`, `NEBRA_PASSWORD`, and `NEBRA_BASE_URL` (the PDS your account actually lives on — skip only if you're on `bsky.social` itself)

Run it

two terminals

- ☐ Terminal A: `./run_producer.sh` — a new simulated reading every 5s, each sent as a real ATProto record
- ☐ Terminal B: `./run_consumer.sh` — connects to the public Jetstream firehose, prints back the same readings within seconds
- ☐ `Ctrl-C` stops either loop for good

✓ Checkpoint

Terminal B's lines match Terminal A's `deviceId / value / unit` — sensor → PDS record → Jetstream → consumer, working end to end over the same public infrastructure Bluesky posts travel over.

REAL (NON-SIMULATED) DATA SOURCES — PICK ONE

RASPBERRY PI OR WEMOS BOARD

Pi: wire a real DS18B20 temperature probe to a GPIO pin (`w1thermsensor`), replace `read_sensor()`'s simulated value — see the Appendix for wiring. **Wemos/ESP8266:** flash `noizetoyz/firmware/esp-wifi/bmp180_wifi/`, then run `live-data/wifi_sensor_relay.py` — same `style.tilde.hacking.sensorReadi` collection, no code changes needed.

WEBCAM

Four readings (brightness, saturation, hue, contrast), no Pi or Python image library needed — `live-data/WEBCAM-SENSORS.md`. On the Toons wall these show as a colored shape instead of a fixed icon: a circle in whatever color the camera actually sees, a black/white square for brightness.

LAPTOP / NO HARDWARE

CPU temperature, weather, ping latency, uptime — already wired into `read_sensor() / UNIT`, just set `SENSOR_TYPE` — `live-data/LOCAL-SENSORS.md`. `./run_local_sensors.sh start` runs these plus all 4 webcam readings at once instead of one at a time — see `live-data/OPS.md`.

SOMETHING ELSE

`SENSOR_TYPE / UNIT` are just strings — relabel to match whatever you're actually measuring.

IF SOMETHING BREAKS

NO MODULE NAMED 'NEBRA'

Not in the pipenv shell / didn't
`pip install nebra`, or Python
<3.11.

NOTHING IN TERMINAL B

Double-check
`NEBRA_HANDLE` / `PASSWORD` in
Terminal A, and both terminals
hit the same Jetstream instance.

401 INVALID IDENTIFIER/PASSWORD

Set `NEBRA_BASE_URL` to your
account's actual PDS — Nebra
defaults to `bsky.social`.

400 ... GOT 23.8

A raw float — AT Protocol has no
float type. This skeleton already
works around it (scaled-integer
`value / valueScale`).

STRETCH GOALS

- ☐ Replace `consumer_viewer.py`'s `print()` with something visual — a terminal sparkline, or a tiny live-charting page
- ☐ Stream a different collection to prove "one substrate" yourself — try `cx.vmx.matadisco` instead of `style.tilde.hacking.sensorReading`
- ☐ Propose a real NSID for the sensor-reading lexicon (currently a placeholder) and say why
- ☐ Watch the same stream with a different tool — `goat firehose --ops -c style.tilde.hacking.sensorReading`
- ☐ Deploy the producer as a Cloudflare Worker on a cron trigger instead of a local script (Matadisco's own `sentinel-to-atproto` pattern)

Play a Live Synth

A small Mozzi-based synth anyone in the room can play — browser or real hardware. Every note-on publishes a real `music.atproto.noizetoyz.synth.note` record; a relay writes it to a PDS and re-broadcasts it to every board in the room.

No hardware? Just play

any laptop/phone

- ☐ Join the `noizetoyz` WiFi network (password `synthbeep`) — or use `hacking.tilde.style/player.html`, which detects and falls back to the local link itself
- ☐ Open `player.html` (or `make-noise.html` for the fuller how-to-play first)
- ☐ Play a note, then try **scrub**, **wavefold**, **filter**, or **fm** — drag each card's XY pad

Got an ESP8266/ESP32/UNO?

real hardware

- ☐ Join the `noizetoyz` WiFi network
- ☐ `cd noizetoyz/firmware && ./flash.sh` — waits for your board, gives a numbered sketch menu
- ☐ Power it up — it plays back everyone else's notes, yours too

✓ Checkpoint

An instant local preview (tone/fold modes only), then moments later hear it played back on a real speaker somewhere in the room. Open `viewer.html` to see your note land as a real record.

IF SOMETHING BREAKS

BOARD NEVER ENUMERATES

Swap the USB cable first — most kit cables are charge-only (LED lights up, never registers as serial).

FLASHES FINE, NO SOUND

Double-check you're on `noizetoyz` WiFi, not your phone's data or the venue's main network.

PLAYER SHOWS NO RELAY ADDRESS

The venue's relay address changes between setups — ask at the station.

STRETCH GOALS

- ☐ Run your own relay: `cd noizetoyz/relay`, set `ATPROTO_HANDLE / PASSWORD`, `./run_relay.sh` — watch with `synth_console_viewer.py`
- ☐ Read `lexicon/music.atproto.noizetoyz.synth.note.json` and propose a real Mozzi patch (filters, a real ADSR envelope) in place of the current `fxType=tremolo` placeholder

□ Full architecture and hardware war stories: `noizetoyz/README.md` , `noizetoyz/OPS.md`

Build a custom feed automation

airglow.run is a no-code "if this ATProto event happens, do that" automation platform — already the plumbing behind hacking.tilde.style's "recently connected via youandme.at" feed. Build your own, and it flies across **toons.tilde.style**, the workshop's big-screen wall — no code, no waiting on anyone else.

Setup

airglow.run

- ☐ Sign in to airglow.run with your ATProto handle
- ☐ Browse airglow.run/u/tilde.style — this workshop's own working toon examples, clonable with one click

Build it

trigger · condition · action

- ☐ Pick a **trigger**: any ATProto record type you care about
- ☐ (Optional) a **condition** — only fire on a field match
- ☐ **Action**: create a record, collection `style.tilde.hacking.toon`, fields `emoji` (required), `label` (optional), `createdAt` = `{{event.commit.record.createdAt}}`
- ☐ Save, enable, trigger it for real

✓ Checkpoint

Your own emoji flies across the top lane of `toons.tilde.style` within a few seconds — purely from an automation *you* built.

IF SOMETHING BREAKS

"LEXICON'S SCHEMA COULD NOT BE RESOLVED"

Harmless — type the condition field path in by hand:

```
event.commit.record.  
<fieldName> (e.g.  
event.commit.record.sensorType)
```

NOTHING SHOWS UP

Double-check the collection name is exactly

`style.tilde.hacking.toon` — a typo won't error, it'll just write somewhere nobody's watching.

WALL SHOWS ✨ INSTEAD OF YOUR EMOJI

The `emoji` field is missing or misnamed — ✨ is the wall's fallback.

STILL NOTHING, AUTOMATION LOOKS RIGHT

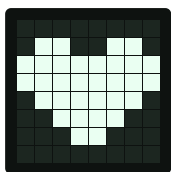
Check it's saved *active* (not a draft), and that you triggered a genuinely new event after saving — an event from before won't retroactively fire it.

STRETCH GOALS

- ☐ Give your automation a real condition — filter to a keyword, author, or field value
- ☐ Chain two automations — a narrow one and a duller catch-all — so specificity shows live on the wall
- ☐ Read `landing-page/lexicon/style.tilde.hacking.toon.json` and `.connection.json` for the pattern already in production use

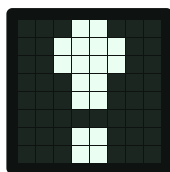
OLED status icons & how they map to Toons

8×8, one bit per pixel — the real constraint of the SSD1306 shield wired to every Noizetoyz board. Every icon here also has an emoji equivalent used on the **toons.tilde.style** wall and in `viewer.html`'s pulse banner, chosen to match — same icon language, hardware screen to big screen.



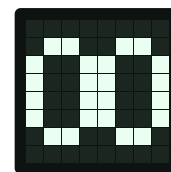
Heart

no Toons emoji
relay config fetched from real
ATProto/HappyView



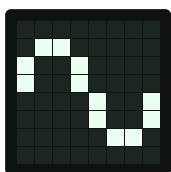
Exclamation

no Toons emoji
`DEFAULT_RELAY_HOST` fallback in
use



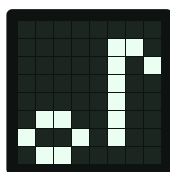
Link

no Toons emoji
downlink connected



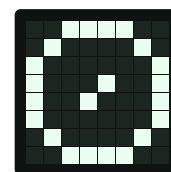
Broken link

no Toons emoji
downlink not connected



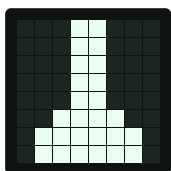
Note-in

→ 🎵 tone / 🗑️ scrub / 🍌
fold / ☕ filter / 📺 fm / 🎸
pluck
a note just arrived over the
downlink



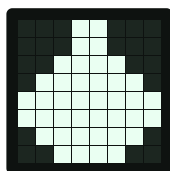
Pressure

→ 🌡️
barometer dial — new 2026-09-04



Temperature

→ 🌡️
same bytes as the Noizetoyz icon
sheet



Humidity

→ 💧
same bytes as the Noizetoyz icon
sheet

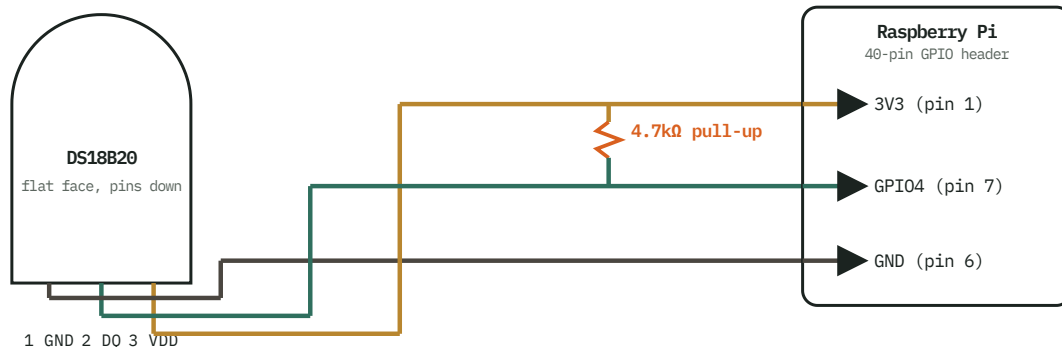
Byte order matches the firmware's own `PROGMEM` arrays: one byte per row, MSB first, top to bottom — reproduced exactly, not redrawn. **Pressure/Temperature/Humidity** also appear in `SENSOR_EMOJI`; **Note-in**'s emoji varies by the note's `mode` (`NOTE_EMOJI`) — see `toons/index.html` and `viewer.html`.

DS18B20 SENSOR WIRING — RASPBERRY PI OR WEMOS

DS18B20

T0-92, flat face toward you, pins down → Raspberry Pi GPIO

— power — data + pull-up — ground

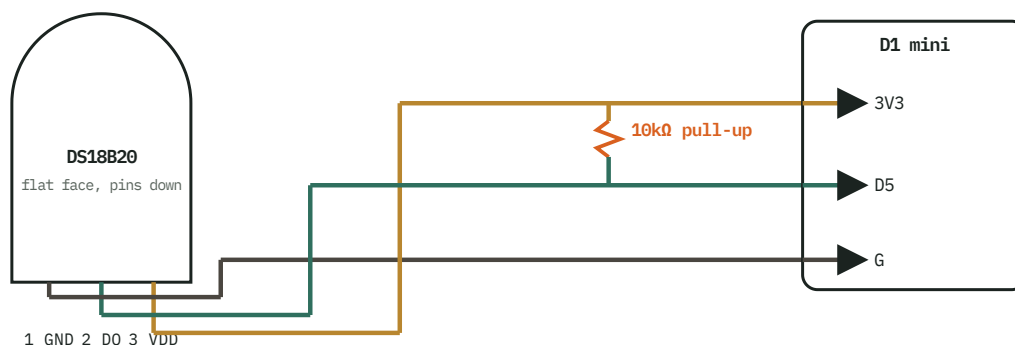


VDD→3V3 (pin 1), GND→GND (pin 6), DQ→GPIO4 (pin 7) — the Pi's default 1-Wire pin, matching `w1thermsensor`'s out-of-the-box config (enable via `raspi-config`'s 1-Wire option, or `dtoverlay=w1-gpio` in `/boot/config.txt`). Datasheet-spec pull-up is 4.7kΩ, bridging VDD and DQ right at the sensor end. Mini breadboard, jumper wires, and a pull-up resistor are all available at the station — no need to bring your own.

DS18B20

T0-92, flat face toward you, pins down → Wemos D1 mini

— power — data + pull-up — ground



VDD→3V3, GND→G, DQ→D5 — same target pins used for a bare DHT22 too, see [noizetoyz/handout/noizetoyz-hardware-handout.html](https://noizetoyz.com/handout/noizetoyz-hardware-handout.html) for that variant. Datasheet-spec pull-up is 4.7kΩ; a 10kΩ from the resistor box works fine too. No ready-made WiFi-networked sketch for DS18B20 yet (`ds18b20_test.ino` is a local-only smoke test) — the Wemos/ESP8266 networked option in the note above (BMP180) is the one that publishes over WiFi out of the box today.

IOSP ATScience workshop, Leiden, October 2026 · compiled from Sensor Tasks, Noizetoyz, and Airglow Toons · code and full instructions: github.com/ATProto-Science/iosp-hacking-stations · live pages: hacking.tilde.style, toons.tilde.style.